

Capítulo 3 — Reglas y conjunciones

Curso de Prolog

Una consulta, dos objetivos

lo que se ingresa, completo

consulta



?- gusta(ana, Que), gusta(luis, Que).

|

objetivo 1

|

y

|

objetivo 2

conjunción: los dos objetivos deben cumplirse

Los gustos de la familia

% gusta(P, C): a P Le gusta C.

`gusta(juan, futbol).`

`gusta(ana, prolog).`

`gusta(ana, futbol).`

`gusta(luis, futbol).`

`gusta(eva, prolog).`

`gusta(sofia, dibujar).`

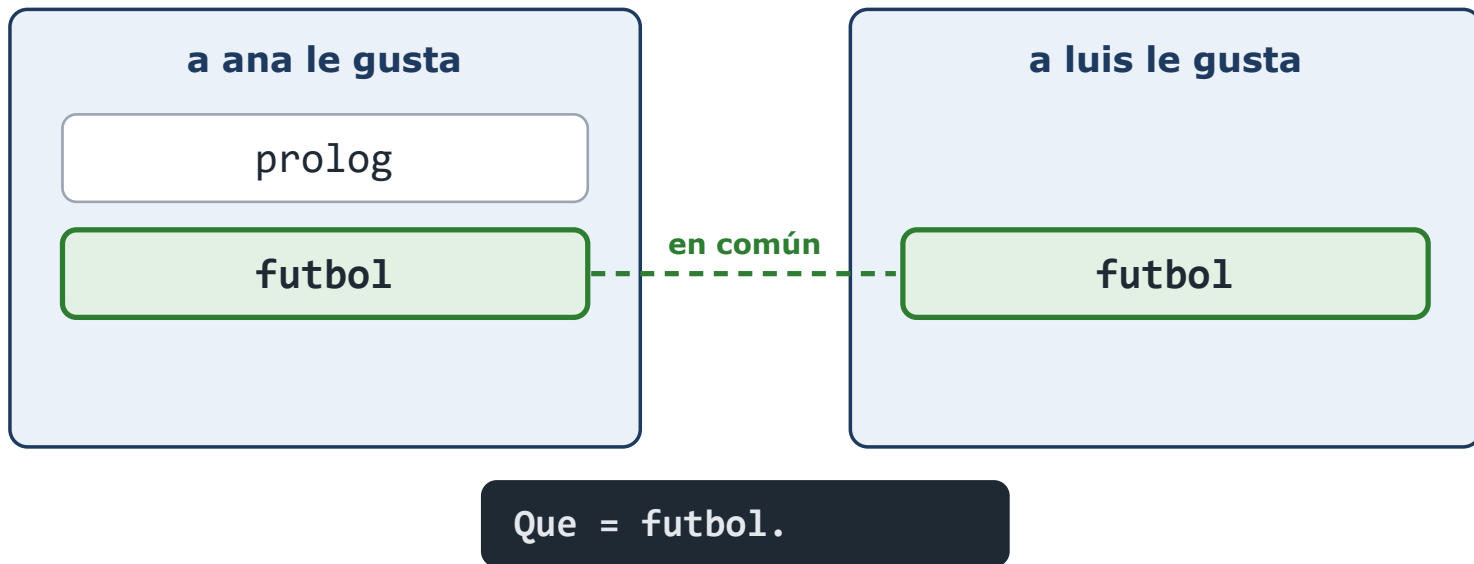
`?- gusta(ana, Que), gusta(luis, Que).`

`Que = futbol.`

Una variable compartida

la misma variable: el mismo valor en los dos objetivos

?- gusta(ana, Que), gusta(luis, Que).



Variables distintas

```
?- gusta(ana, Una), gusta(luis, Otra).
```

```
Una = prolog,
```

```
Otra = futbol ;
```

```
Una = futbol,
```

```
Otra = futbol.
```

Una consulta que falla (1)

?- `gusta(sofia, Que), gusta(ana, Que).`

se busca `gusta(sofia, Que)`

<code>gusta(juan, futbol).</code>	✗
<code>gusta(ana, prolog).</code>	✗
<code>gusta(ana, futbol).</code>	✗
<code>gusta(luis, futbol).</code>	✗
<code>gusta(eva, prolog).</code>	✗
<code>gusta(sofia, dibujar).</code>	✓

Que

dibujar

El primer objetivo se prueba con el último hecho: Que queda ligada a dibujar.

estados de Que



Una consulta que falla (2)

?- `gusta(sofia, Que), gusta(ana, Que).`

Fail

se busca `gusta(ana, dibujar)`

<code>gusta(juan, futbol).</code>	×
<code>gusta(ana, prolog).</code>	×
<code>gusta(ana, futbol).</code>	×
<code>gusta(luis, futbol).</code>	×
<code>gusta(eva, prolog).</code>	×
<code>gusta(sofia, dibujar).</code>	×

Que

dibujar

Con Que ligada, el segundo objetivo es `gusta(ana, dibujar)`.

Ningún hecho lo prueba: falla.

estados de Que



Una consulta que falla (3)

?- `gusta(sofia, Que), gusta(ana, Que).`

Redo

se busca `gusta(sofia, Que)`

`gusta(juan, futbol).`

`gusta(ana, prolog).`

`gusta(ana, futbol).`

`gusta(luis, futbol).`

`gusta(eva, prolog).`

`gusta(sofia, dibujar).`

Que

libre

Se retrocede al primer objetivo:
Que queda libre otra vez.
No quedan hechos por examinar.

false.

estados de Que



Una consulta con éxito (1)

?- `gusta(ana, Que), gusta(luis, Que).`

se busca `gusta(ana, Que)`

Que

prolog

`gusta(juan, futbol).`



`gusta(ana, prolog).`



`gusta(ana, futbol).`

← continúa aquí

`gusta(luis, futbol).`

`gusta(eva, prolog).`

`gusta(sofia, dibujar).`

Que queda ligada a prolog.
Quedan hechos sin examinar:
se registra dónde continuar.

estados de Que

libre



prolog

Una consulta con éxito (2)

?- `gusta(ana, Que), gusta(luis, Que).`

Fail

se busca `gusta(luis, prolog)`

<code>gusta(juan, futbol).</code>	×
<code>gusta(ana, prolog).</code>	×
<code>gusta(ana, futbol).</code>	×
<code>gusta(luis, futbol).</code>	×
<code>gusta(eva, prolog).</code>	×
<code>gusta(sofia, dibujar).</code>	×

Que

prolog

El segundo objetivo es
`gusta(luis, prolog).`
Ningún hecho lo prueba: falla.

estados de Que



Una consulta con éxito (3)

?- `gusta(ana, Que), gusta(luis, Que).`

Redo

se busca `gusta(ana, Que)`

`gusta(juan, futbol).`

`gusta(ana, prolog).`



`gusta(ana, futbol).`



← continúa aquí

`gusta(luis, futbol).`

`gusta(eva, prolog).`

`gusta(sofia, dibujar).`

Que

futbol

Se retrocede al primer objetivo y se continúa donde quedó. Que se desliga y queda ligada a futbol.

estados de Que



Una consulta con éxito (4)

?- `gusta(ana, Que), gusta(luis, Que).`

se busca `gusta(luis, futbol)`

- `gusta(juan, futbol).` ✗
- `gusta(ana, prolog).` ✗
- `gusta(ana, futbol).` ✗
- `gusta(luis, futbol).` ✓
- `gusta(eva, prolog).`
- `gusta(sofia, dibujar).`

Que

`futbol`

`gusta(luis, futbol)` está en el programa: los dos objetivos se cumplen a la vez.

`Que = futbol.`

estados de Que



El orden de los objetivos

?- gusta(ana, Que), gusta(luis, Que).

1 gusta(ana, Que) ✓

Que = prolog

2 gusta(luis, prolog) ✗

falla: se retrocede

3 gusta(ana, Que) ✓

Que = futbol

4 gusta(luis, futbol) ✓

se cumple

Que = futbol.

un retroceso

?- gusta(luis, Que), gusta(ana, Que).

1 gusta(luis, Que) ✓

Que = futbol

2 gusta(ana, futbol) ✓

se cumple

Que = futbol.

ningún retroceso

la misma respuesta, con menos trabajo

Una regla

```
%! es_padre(?P) is nondet.
```

```
%
```

```
% P es padre de alguien. Una regla de una sola condición.
```

```
es_padre(P) :-
```

```
    padre(P, _).
```

Cabeza y cuerpo

cabeza

lo que se concluye

es_padre(P)

: -

cuerpo

lo que debe cumplirse

padre(P, _).

si

alguien:

el valor no interesa

P es padre si P es padre de alguien

Respuestas repetidas

```
?- es_padre(Quien).
```

```
Quien = juan ;
```

```
Quien = juan ;
```

```
Quien = pedro ;
```

```
Quien = pedro.
```

padre(juan, ana).

juan

padre(juan, pedro).

juan

padre(pedro, luis).

pedro

padre(pedro, eva).

pedro

Una respuesta por cada demostración:
juan es padre de ana y de pedro.

Varias cláusulas: «o»

```
progenitor(P, H) :-
```

```
    padre(P, H).
```

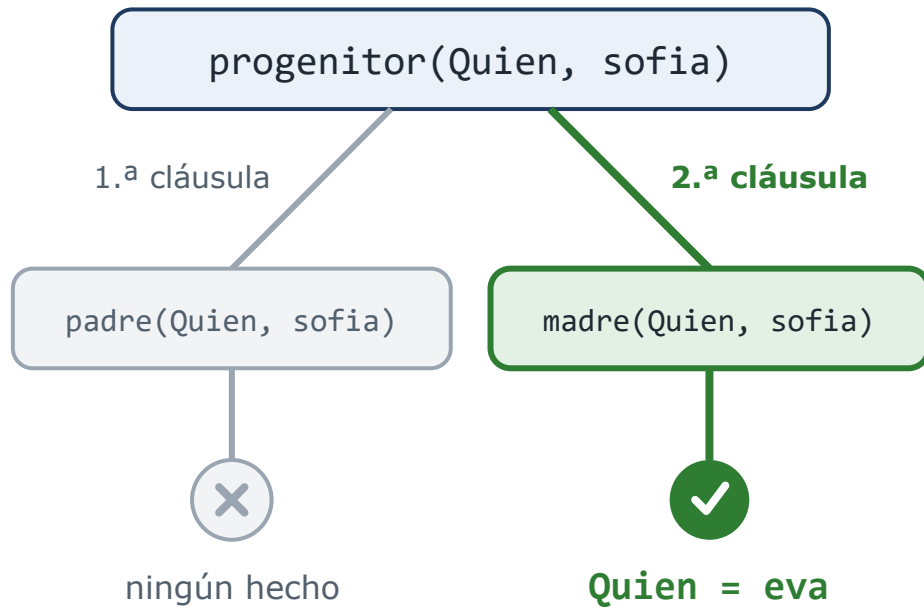
```
progenitor(P, H) :-
```

```
    madre(P, H).
```

```
?- progenitor(Quien, sofia).
```

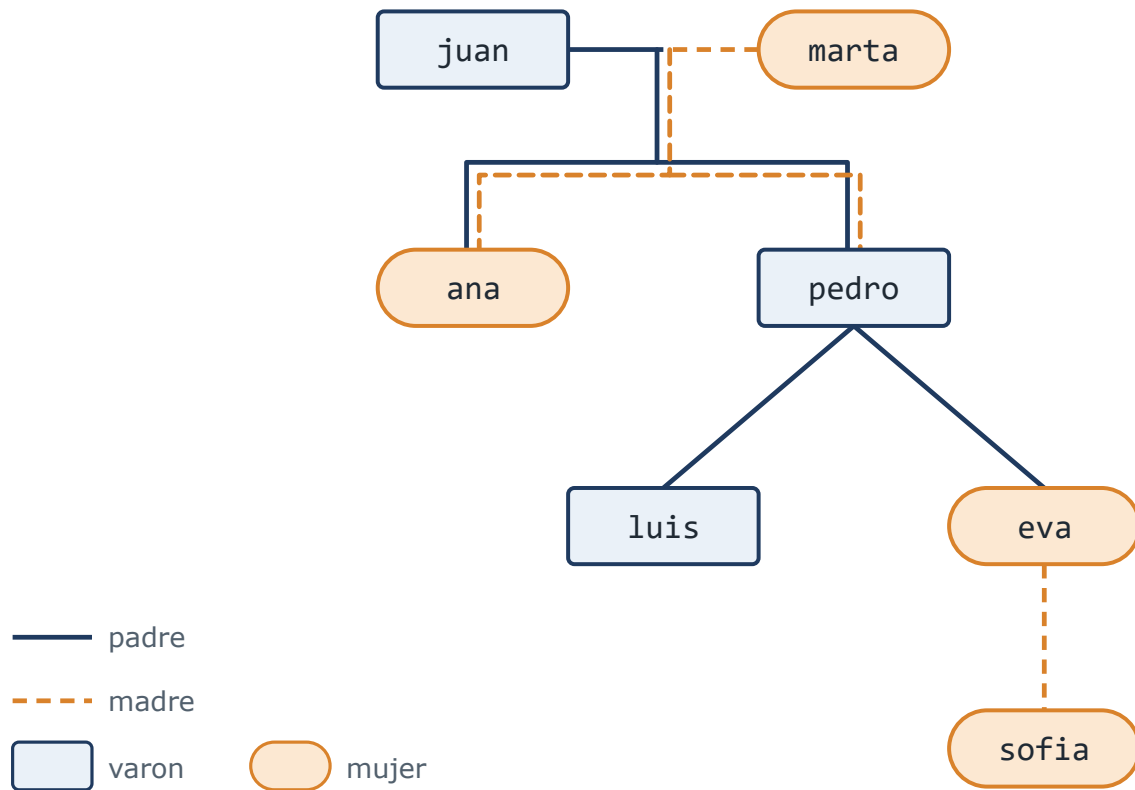
```
Quien = eva.
```

Una cláusula, después la otra



Las cláusulas se prueban
en el orden en que están escritas.

La familia



Encadenar dos relaciones

```
%! abuelo(?A, ?N) is nondet.
```

```
%
```

```
% A es el abuelo de N.
```

```
abuelo(A, N) :-  
    varon(A),  
    progenitor(A, P),  
    progenitor(P, N).
```

```
%! abuela(?A, ?N) is nondet.
```

```
%
```

```
% A es la abuela de N.
```

```
abuela(A, N) :-  
    mujer(A),  
    progenitor(A, P),  
    progenitor(P, N).
```

Cuatro plantillas

Plantilla 2 — Derivar una relación de otra

```
nueva(X) :-  
    vieja(X, _).
```

Plantilla 3 — Definir por casos

```
p(X) :-  
    primer_caso(X).  
p(X) :-  
    otro_caso(X).
```

Plantilla 4 — Encadenar dos relaciones

```
nueva(A, C) :-  
    primera(A, B),  
    segunda(B, C).  
  
B: la misma  
en los dos
```

Plantilla 5 — Generar y después comprobar

```
nueva(X) :-  
    candidato(X),  
    condicion(X).
```

El alcance de una variable

una cláusula

```
abuelo(A, N) :-  
    varon(A),  
    progenitor(A, P),  
    progenitor(P, N).
```

la misma P: la misma persona

otra cláusula

```
progenitor(P, H) :-  
    padre(P, H).
```

otra P, aunque se llame igual

Cada uso de una regla emplea variables nuevas

`progenitor(A, P) → progenitor(P1, H1) :- padre(P1, H1).`

`progenitor(P, N) → progenitor(P2, H2) :- padre(P2, H2).`

abuelo(juan, Quien), paso a paso (1)

?- abuelo(juan, Quien).

```
abuelo(A, N) :-
```

```
    varon(A),
```

```
    progenitor(A, P),
```

```
    progenitor(P, N).
```

subobjetivos

abuelo(juan, Quien) con abuelo(A, N)

A = juan

N y Quien: la misma variable

variables

A

juan

P

libre

N

libre

Quien

libre

la misma
variable

La consulta unifica con la
cabeza de la regla.

abuelo(juan, Quien), paso a paso (2)

?- abuelo(juan, Quien).

abuelo(A, N) :-



varon(A),



progenitor(A, P),

progenitor(P, N).

subobjetivos

progenitor(juan, P)

1.ª cláusula: padre(juan, P)

padre(juan, ana) → P = ana

variables

A

juan

P

ana

N

libre

Quien

libre

la misma
variable

varon(juan) se cumple.

El primer hijo de juan es ana.

abuelo(juan, Quien), paso a paso (3)

?- abuelo(juan, Quien).

abuelo(A, N) :-



varon(A),



progenitor(A, P),

progenitor(P, N).



subobjetivos

progenitor(ana, N)

padre(ana, N): ningún hecho

madre(ana, N): ningún hecho

retroceso: padre(juan, pedro)

P = pedro

variables

A

juan

P

pedro

N

libre

Quien

libre

la misma
variable

ana no tiene hijos en el programa:
se retrocede y P pasa a pedro.

abuelo(juan, Quien), paso a paso (4)

?- abuelo(juan, Quien).

abuelo(A, N) :-

varon(A),

progenitor(A, P),

progenitor(P, N).

subobjetivos

progenitor(pedro, N)

1.ª cláusula: padre(pedro, N)

padre(pedro, luis) → N = luis

variables

A

juan

P

pedro

N

luis

Quien

luis

la misma
variable

N queda ligada a luis, y con ella
Quien: son la misma variable.

Quien = luis ;

Los nietos de juan

```
?- abuelo(juan, Quien).  
Quien = luis ;  
Quien = eva ;  
false.
```

Variables que aparecen una sola vez

```
abuelo(A, N) :-  
    varon(A),  
    progenitor(A, P),  
    progenitor(Q, N).
```

nada une los dos
objetivos progenitor

P aparece una vez

Q aparece una vez

Warning: reglas.pl:54:

Warning: Singleton variables: [P,Q]

el nombre está mal escrito

corregirlo

el valor no interesa

—

no interesa, y el nombre informa

_Hijo

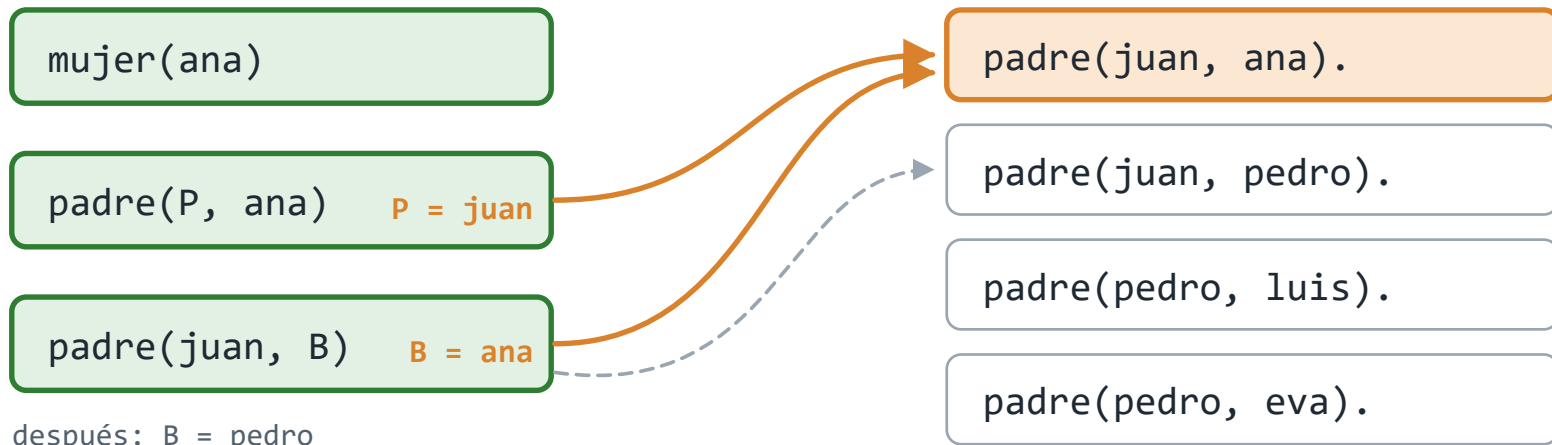
Una regla con una respuesta de más

```
hermana(A, B) :-  
    mujer(A),  
    padre(P, A),  
    padre(P, B).
```

```
?- hermana(ana, Quien).  
Quien = ana ;  
Quien = pedro.
```

El mismo hecho, dos veces

?- hermana(ana, Quien).



```
Quien = ana ;  
Quien = pedro.
```

El mismo hecho prueba los dos objetivos padre: B queda ligada a ana, la misma persona que A.

La condición que faltaba

```
%! hermana_de_verdad(?A, ?B) is nondet.
```

```
%
```

```
% A es hermana de B, y no son la misma persona.
```

```
hermana_de_verdad(A, B) :-
```

```
    mujer(A),
```

```
    padre(P, A),
```

```
    padre(P, B),
```

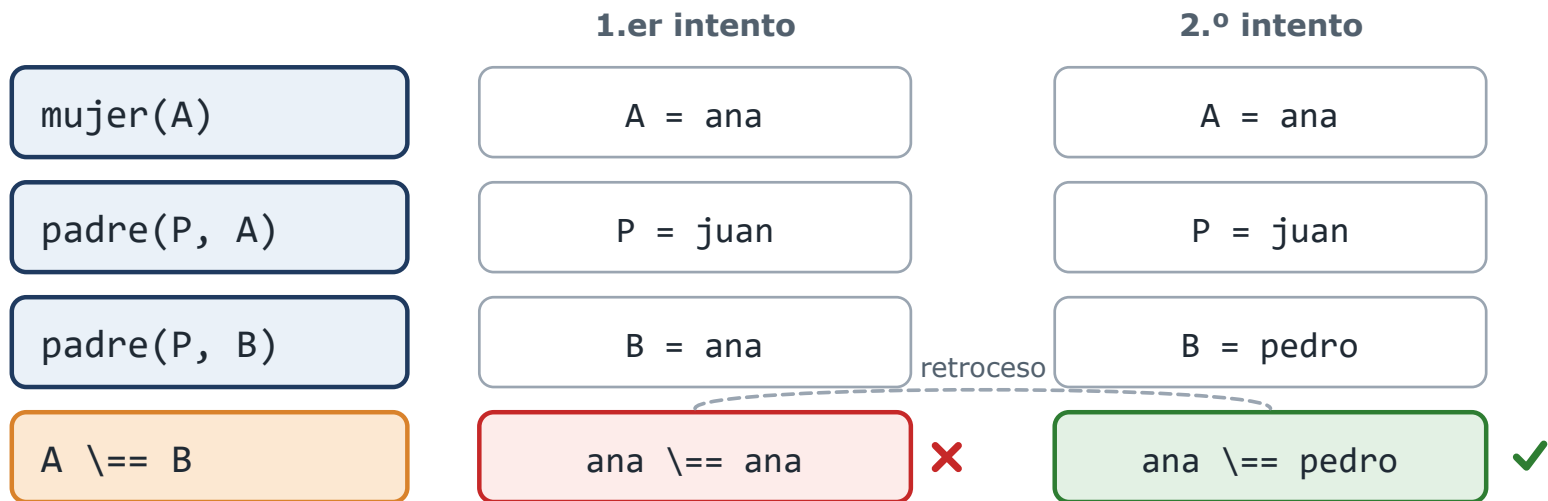
```
    A \== B.
```

```
?- hermana_de_verdad(ana, Quien).
```

```
Quien = pedro.
```

La comparación, al final

?- hermana_de_verdad(ana, Quien).



Quien = pedro.

Al final, A y B ya tienen valor:
la comparación descarta a ana.

La primera prueba

% Respuestas de la regla incompleta: ana se incluye a sí misma.

```
test(hermana_se_cuenta_a_si_misma, all(Q == [ana, pedro])) :-  
    hermana(ana, Q).
```

% Respuestas esperadas.

```
test(hermanas_de_verdad, all(Q == [pedro])) :-  
    hermana_de_verdad(ana, Q).
```

```
test(nadie_es_hermana_de_si_misma, [fail]) :-  
    hermana_de_verdad(eva, eva).
```

% pedro es varón: no es hermana de nadie, aunque tenga hermanos.

```
test(pedro_no_es_hermana, [fail]) :-  
    hermana_de_verdad(pedro, _).
```

Ejecutar las pruebas

se cargan juntos el programa y sus pruebas

```
swipl -g "consult(['hermana.pl','hermana.plt']),run_tests" -t halt
```

```
% [1/5] hermana:hermana_se_cuenta_a_si_misma ..... passed
% [2/5] hermana:hermanas_de_verdad ..... passed
% [3/5] hermana:eva_es_hermana_de_luis ..... passed
% [4/5] hermana:nadie_es_hermana_de_si_misma ..... passed
% [5/5] hermana:pedro_no_es_hermana ..... passed
% All 5 tests passed
```



La salida real muestra además el tiempo de cada prueba.

Resumen

,

y

conjunción: todos los objetivos, con los mismos valores

:-

si

separa la cabeza de una regla de su cuerpo

p :- ...

o

varias cláusulas del mismo predicado: alternativas

\==

no es

los dos términos no son el mismo

plunit: all(... == [...]) todas las respuestas · [fail] debe fallar · [nondet] deja alternativas